

For every token the model has to learn:

loss = $-\log(\text{probability the model gave to the correct token})$

The loss reported during training is the **average** of that number over all the learned tokens in a batch. The name for this is **cross-entropy loss**. The trainer calculates it automatically.

Training needs **one number** that says how wrong the model is, so it can make that number smaller. That number is the **loss**.

Step 1: the model guesses the next token

The model has seen `<tool_call>{"name": "`. Now it must guess the next token.

It doesn't pick just one. It gives **every possible token a probability**, and they add up to 100%:

Possible next token	Probability
<code>get_nav</code>	66%
<code>get_portfolio</code>	24%
<code>start_sip</code>	10%

Step 2: check the probability of the correct token

The training data says the correct token is `get_nav`. The model gave it **66%**. That is the only number that matters for this position.

Step 3: turn that probability into a penalty

The formula is **penalty = $-\log(\text{probability})$** .

You only need what the formula does:

Probability given to the correct token	Penalty
99%	0.01 (almost none)
66%	0.42
50%	0.69
10%	2.30

Probability given to the correct token	Penalty
1%	4.61 (large)

- **Right and sure:** the penalty is close to 0.
- **Right but unsure:** a medium penalty.
- **Gave the right token almost no chance:** a large penalty.

Here, **penalty** just means **a number that measures how wrong the model was on one token**:

- **Bigger number** → more wrong.
- **0** → perfectly right.

It is the same thing as the loss, but for **one token**. The "loss" you see during training is the **average** of these per-token numbers.

How the number is decided:

1. The model gives the correct token some probability, for example 66%.
2. The formula **$-\log(\text{probability})$** turns that into a number:
 - 99% → **0.01** (almost no penalty: right and sure)
 - 66% → **0.42**
 - 10% → **2.30** (big penalty: the right answer was given little chance)
3. Training changes the model's numbers to make these penalties smaller, which means making it surer of the correct tokens.

So "penalty" is just a plain word for **the loss on one token**. In class you can say "error score" instead, if that's clearer for your audience.

Step 4: do this for every token in the answer, then average

The answer is several tokens long. Repeat steps 1–3 at each position, then take the average:

Position	Correct token	Probability given	Penalty
1	get_nav	66%	0.42
2	fund_name	90%	0.11
3	stop token	20%	1.61
		Average = loss	0.71

Only the **answer** tokens are counted. The prompt tokens, marked "ignore", are skipped.

Step 5: make the loss smaller

The trainer works out which way to nudge each trainable number so that the correct tokens get **higher** probabilities. It nudges them a tiny bit, then repeats with the next batch of examples.

As the probabilities of the correct tokens rise, the penalties shrink, and the loss goes down.

Note

Loss = the average penalty for not being sure of the correct next token. Training nudges the model until it is surer of the right tokens, so the loss falls.